ETAS RTA-BSW v12.7.0

RTA-Lin Stack Reference Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-Lin Stack Reference Guide R01 EN – 05.2025

Contents

1 Lin Interface . . . 5
- 1.1 Introduction . . . 5
- 1.2 Supported Features . . . 7
- 1.3 Extensions . . . 9
- 1.4 Limitations . . . 15
- 1.5 Exclusions . . . 16
- 1.6 Deviations . . . 16
- 1.7 API Definition . . . 19
- 1.8 Configuration Advice . . . 20
- 1.9 Integration Advice . . . 25
- 1.10 Hardware Requirements . . . 27

2 Lin State Machine . . . 28
- 2.1 Introduction . . . 28
- 2.2 Supported Features . . . 29
- 2.3 Extensions . . . 30
- 2.4 Limitations . . . 32
- 2.5 Exclusions . . . 32
- 2.6 Deviations . . . 33
- 2.7 API Definition . . . 34
- 2.8 Configuration Advice . . . 34
- 2.9 Integration Advice . . . 35
- 2.10 Hardware Requirements . . . 36

3 Lin Transport Protocol . . . 37
- 3.1 Introduction . . . 37
- 3.2 Supported Features . . . 38
- 3.3 Extensions . . . 38
- 3.4 Limitations . . . 39
- 3.5 Exclusions . . . 39
- 3.6 Deviations . . . 39
- 3.7 API Definition . . . 40
- 3.8 Configuration Advice . . . 40
- 3.9 Integration Advice . . . 40
- 3.10 Hardware Requirements . . . 41

4 Glossary 42

1 Lin Interface

Module name	LinIf
Package	RTA-LIN
AUTOSAR Module ID	62
AUTOSAR Version	22-11

1.1 Introduction

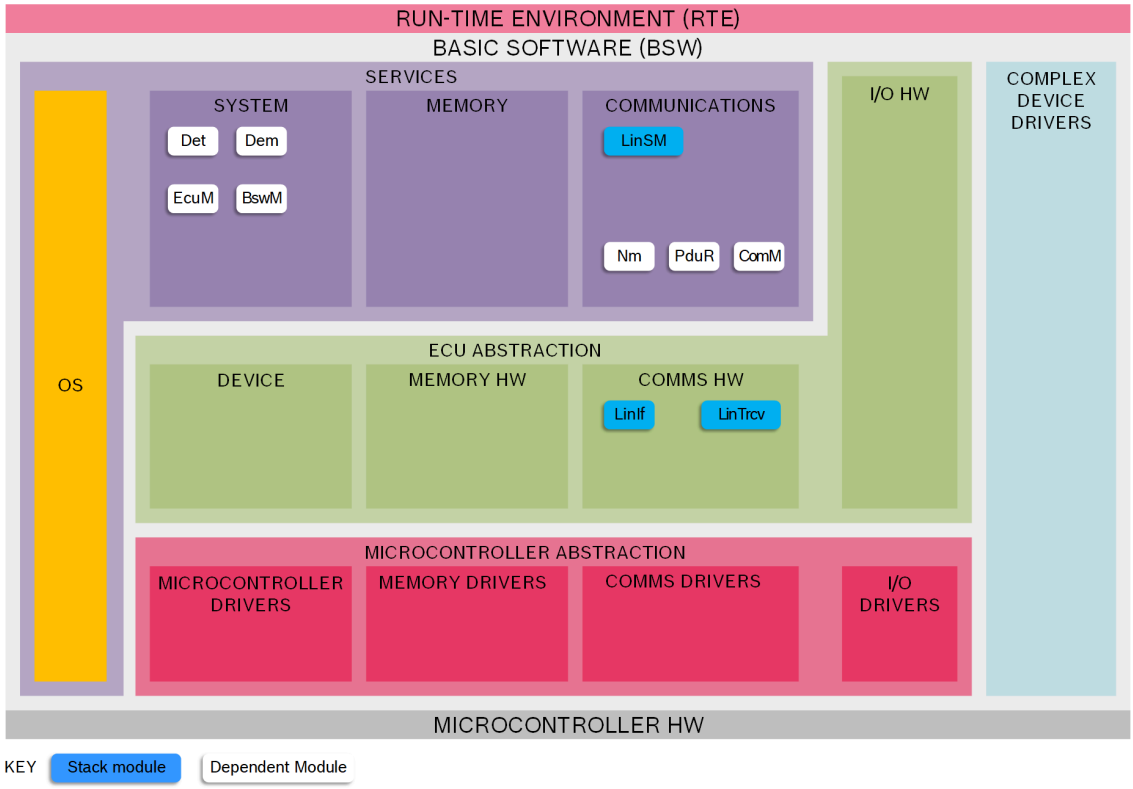

Figure 1.1.RTA-LIN Architecture

The LIN stack is used for in-vehicle communication that uses LIN devices (e.g. wiper control, roof control etc).

The LIN Interface module (LinIf) is used as an interface for routing PDUs between the PDU Router (PduR) and the LIN Driver (Lin). LinIf is designed to be hardware independent.

The diagram above shows the position of LinIf in the AUTOSAR layered architecture.

1.1.1 LinTp

LinTp is part of the AUTOSAR-defined LinIf module - see the separate LinTp section (Section 3.1) for more details.

1.1.2 Interaction and Dependencies

LinIf has interaction and dependencies with upper layer and lower-layer modules, as described in the two sub-sections below.

1.1.2.1 Upper layers

PduR and CDD

LinIf connects to the PduR (and/or alternative upper modules, e.g. CDDs) for the transmission and reception of frames. It is assumed that these upper modules are responsible for the copying of the data of the frames for reception and transmission.

LinSM

LinIf connects to LinSM (and/or alternative upper modules, e.g. CDDs) which are responsible for the control flow of the whole LIN stack. LinIf's interaction with LinSM includes the following -

- LinSM forwards Schedule Table Requests to LinIf.
- LinSM requests the transmission of wake-up and Go-to-Sleep commands.

1.1.2.2 Lower layers

Lin

LinIf's dependency on the LIN Driver includes the following:

- Transmission of the header and response parts of a Frame.
- Transmission of the Go-to-Sleep and Wakeup commands.
- Query of reception of the response part of a Frame.

LinIf does not use or access the LIN hardware or assume information about it any way other than that which is provided by the LIN Driver through function calls.

LinTrcv

LinIf maps the following services for all underlying LIN Transceiver Drivers to one unique interface:

- Unique LIN Transceiver Driver mode request and read services to manage the operation modes of each underlying LIN transceiver device.
- Read service for LIN Transceiver Wakeup reason support.
- Mode request service to enable/disable/clear the Wakeup event state of each LIN Transceiver used.

LinIf does not use or access the LIN hardware or assume information about it any way other than that which is provided by the LIN Transceiver Driver through function calls.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

1.2 Supported Features

1.2.1 General Summary

- Frame Reception and Transmission
- Schedule Table Manager
- LIN Interface State-Machine
- LIN Channel Sub-State-Machine
- Go-to-sleep process
- Wakeup process
- Wakeup Confirmation
- Diagnostic Transport layer (LinTp)
- Multiple Channels

1.2.2 Schedule Table handling

The following occurs when a schedule table is already running and a new schedule request is made for the same schedule table:

- For RUN_CONTINUOUS, no action is performed. A schedule table confirmation is given.
- For RUN_ONCE, the new RUN_ONCE is flagged to be executed after the current RUN_ONCE has completed.

While a schedule table is running, any new schedule table requests will remain in a pending status. If there is more than one request pending when the currently running request completes, the most recent pending request will be the one that is processed.

The exact time when schedule tables are switched depends on the optional configuration parameter LinIfScheduleChangeNextTimeBase. If it is disabled, the schedule table is switched when the next entry of a RUN_CONTINUOUS schedule table is due, or when the last entry of a RUN_ONCE schedule table is finished. If LinIfScheduleChangeNextTimeBase is enabled, LinIf will select the new schedule table directly after message transmission or reception within an entry has been completed, ensured by status checks for transmission and reception.

No schedule table priority handling is done within LinIf, although it is possible within LinSM.

If a running schedule table is interrupted by a NULL_SCHEDULE schedule table, or if a RUN_CONTINUOUS schedule table is interrupted by another RUN_CONTINUOUS schedule table, a schedule switch will take place. When the first schedule table is switched back it will start from the beginning and not resume from the

interrupted location. (LinIfResumePosition is ignored).

1.2.3 Transmission of multiple sleep frames

It is possible to transmit more than one sleep frame when LinIf_GotoSleep is requested by LinSM. The number of sleep frames that are transmitted is a channel-specific configuration parameter. This feature is required because some of the slaves do not understand a single sleep frame transmitted on the bus.

1.2.4 LinSlave Support

LinIf and LinSM support the LinSlave feature.

- The LinIf configuration works only as a pure master or a pure slave, combination of both master and slave configurations are not supported.
- LinIf and LIN channels have a one-to-one mapping for slave nodes.
- Sleep, wakeup and bus idle conditions are supported as per AUTOSAR R21-11.
- Event triggered frames are supported only in slave configurations.
- Sporadic frames are supported only in master nodes, which is as per AUTOSAR.
- LinSlave is supported in the Lin upper layers - LinIf, LinSM and ComM - but not in the Lin Drivers.

1.2.5 Non-AUTOSAR Features in LinIf

In addition to the AUTOSAR-defined features described above, LinIf also provides the following additional functionality.

1.2.5.1 LinIf_Tp_InitChannel

It is possible to reset the TP channel state to IDLE on request. This is done with the LinIf_Tp_InitChannel extension API.

1.2.5.2 LinIfScheduleChangeNextTimeBase

The LinIfScheduleChangeNextTimeBase extension parameter enables/disables the switch to a new schedule table at the start of the next time base after status check. When LinIfScheduleChangeNextTimeBase is set to TRUE, LinIf selects a new schedule table in the next main function. The default value is set to FALSE and the schedule table switch happens when the schedule entry is due.

1.2.5.3 Support for Wildcards

The ReadByIdentifier service in Node configuration services supports all the wildcards as per ISO17987-3:2016, Including the wildcard value for NAD, Supplier ID and Function ID.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 LinIf/LinIfGeneral/LinIfErrorRecovery

Short Name	LinIfErrorRecovery
Description	Enables/Disables the LINIF_ERROR_RECOVERY_SUPPORT.If LINIF_ERROR_RECOVERY_SUPPORT is enabled, Monitoring hooks enabled.If LINIF_ERROR_RECOVERY_SUPPORT is disabled, Monitoring hooks disabled.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.2 LinIf/LinIfGeneral/LinIfRbForeignDrvSupport

Short Name	LinIfRbForeignDrvSupport
Description	Enable third party driver support.
Introduction	When enabled, the callback APIs of LinIf will be named LinIf_Integn_{CallbackAPIName} and also re-routes driver API calls to integration code. When this configuration parameter is enabled, the template files LinIf_Integn.c_tpl and LinIf_Integn.h_tpl shall be used for call adaptation between LinIf and drivers.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.3 LinIf/LinIfGeneral/LinIfRbLastEntryOfSchelTblConfrm

Short Name	LinIfRbLastEntryOfSchelTblConfrm
Description	Switches the Schedule table end notification callback ON or OFF.Schedule table end notification callback functions are enabled when this parameter is set to trueand disabled when set to false. * true: enabled (ON).* false: disabled (OFF).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.4 LinIf/LinIfGeneral/LinIfRbScheduleClbkSupport

Short Name	LinIfRbScheduleClbkSupport
Description	States if the Node configuration Auto Addressing feature is supported. Only applicable for Master Nodes
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.5 LinIf/LinIfGeneral/LinIfidCalibrationSupport

Short Name	LinIfidCalibrationSupport
Description	Switches the Calibration of LinIf id (The id for Lin frames is provided from calibration tool) ON or OFF. * true: enabled (ON).* false: disabled (OFF).
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.6 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfChannelId

Short Name	LinIfChannelId
Description	This parameter holds the unique channel index value. The value shall be the same as the ComMChannelId of the ComM-Channel referenced by LinIfComMNetworkHandleRef.Implementation Type: NetworkHandleType
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.7 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfDiagInterleavedModesupport

Short Name	LinIfDiagInterleavedModesupport
Description	This parameter enables/disables the support of Diagnostic Interleaved Mode
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.8 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfDriverRef

Short Name	LinIfDriverRef
Description	Reference to the LIN Driver
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

1.3.1.9 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfLength

Short Name	LinIfLength
Description	Length of the LIN SDU in bytes.
Multiplicity	1..1
Value Range	1..8
Default Value	false
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.10 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfPduDirection/LinIfRxPdu/LinIfUserTargetRxPduId

Short Name	LinIfUserTargetRxPduId
Description	Defines the target Id of the CDD for Rx Pdus.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.11 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfPduDirection/LinIfTxPdu/LinIfUserTargetTxPduId

Short Name	LinIfUserTargetTxPduId
Description	Defines the target Id of the CDD for Tx Pdus.
Multiplicity	0..1
Value Range	0..INF
Default Value	false
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.12 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfGotoSleepConfirmationULName

Short Name	LinIfGotoSleepConfirmationULName
Description	This parameter defines the name of the "User_GotoSleepConfirmation()" LinIfGotoSleepConfirmationULName
Introduction	Only used for LIN Slave nodes, ignored for master nodes.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.13 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfGotoSleepIndicationULName

Short Name	LinIfGotoSleepIndicationULName
Description	This parameter defines the name of the "User_GotoSleepIndication()" LinIfGotoSleepConfirmationULName
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.14 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeConfigurationResponseULName

Short Name	LinIfNodeConfigurationResponseULName
Description	This parameter defines the name of the "User_NodeConfigurationResponse()" LinIfNodeConfigurationResponseULName
Introduction	Only used for LIN Master nodes, ignored for Slave nodes.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.15 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfResponseErrorSignalFrameRef

Short Name	LinIfResponseErrorSignalFrameRef
Description	Reference to the response error signal frame.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

1.3.1.16 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfRbDiagMonHookSupport

Short Name	LinIfRbDiagMonHookSupport
Description	Enables or disables the Monitoring Hook function notification updation in case of Diagnostic frames.
Introduction	True: LinIf selects a new schedule table in next main function. Only applicable for LIN Master nodes.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.17 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfScheduleRequestConfirmationULName

Short Name	LinIfScheduleRequestConfirmationULName
Description	This parameter defines the name of the "User_ScheduleRequestConfirmation()" LinIfScheduleRequestConfirmationULName.
Introduction	Only used for LIN Slave nodes, ignored for master nodes.
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

1.3.1.18 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfTransceiverDrvConfig/LinIfTrcvWakeupNotification

Short Name	LinIfTrcvWakeupNotification
Description	Selects whether wakeup indication notification is supported.True: EnabledFalse: Disabled
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.19 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfWakeupConfirmationULName

Short Name	LinIfWakeupConfirmationULName
Description	This parameter defines the name of the "User_WakeupConfirmation()" LinIfWakeupConfirmationULName
Introduction	Only used for LIN Slave nodes, ignored for master nodes.
Multiplicity	0..1
Default Value	-

Type	ECUC-FUNCTION-NAME-DEF

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.2.1 LinIf_DeInit

Syntax	`LinIf_DeInit(void)`
Description	Deinitializes the LIN Interface.

1.3.2.2 LinIf_MonHook_E_OK

Syntax	`LinIf_MonHook_E_OK ( uint8 FrameType, uint8 Direction, PduIdType PduID, Lin_StatusType FrameStatus, uint8 PID, NetworkHandleType Channel )`
Description	This function can only be used when the LinIfErrorRecovery parameter (in LinIf/LinIfDemSupport/) is configured as TRUE. The “FrameType”, “Direction”, “PduID”, “FrameStatus” and “PID” parameters are defined for compatibility but are not used.

1.3.2.3 LinIf_MonHook_E_NOT_OK

Syntax	`LinIf_MonHook_E_NOT_OK ( uint8 FrameType, uint8 Direction, PduIdType PduID, Lin_StatusType FrameStatus, uint8 PID, NetworkHandleType Channel )`
Description	This function can only be used when the LinIfErrorRecovery parameter (in LinIf/LinIfDemSupport/) is configured as TRUE. The “FrameType”, “Direction”, “PduID”, “FrameStatus” and “PID” parameters are defined for compatibility but are not used.

1.3.2.4 LinIf_Diag_MonHook

Syntax	`LinIf_Diag_MonHook( Lin_FramePidType PID, NetworkHandleType Channel,Lin_StatusType FrameStatus)`
Description	This function can only be used when the LinIfRbDiagMonHookSupport parameter (in LinIf/LinIfGlobalConfig/LinIfChannel/) is configured as TRUE. The “FrameStatus” and “PID” parameters are defined for compatibility but are not used.

1.3.2.5 LinIf_LastEnrtyProcessed

Syntax	`LinIf_LastEnrtyProcessed (NetworkHandleType Channel, LinIf_SchHandleType Schedule)`

Description	This function can only be used when the LinIfRbLastEntryOfSchelTblConfrm parameter (in LinIf/LinIfGeneral/) is configured as TRUE. The “Channel” and “Schedule” parameters are defined for compatibility but are not used.

1.3.2.6 LinIf_GetQueStatus

Syntax	`LinIf_GetQueStatus( VAR(NetworkHandleType, AUTOMATIC) LinIf_Channel_u8)`
Description	This function reads the run once queue if it is free returns with LINIF_E_QUEUE_OCCUPIED. This function should be called by LinSM only when Queue Event Handler is enabled in both LinIf and LinSM.

LinIf_Tp_InitChannel

Syntax	`LinIf_Tp_InitChannel( VAR(NetworkHandleType, AUTOMATIC) LinIf_Channel_u8)`
Parameters In	`LinIf_Channel_u8` Channel Id value.
Description	Initializes the LIN TP.

1.4 Limitations

The following AUTOSAR-defined features for LinIf are not supported:

- Slave-to-Slave communication
- Event triggered frames
- Collision-resolving schedule table
- Multiple driver/trcv support

Diagnostics only mode is not supported according to ISO 17987.

1.4.1 LinIfResumePosition

If a running schedule table is interrupted by a NULL_SCHEDULE schedule table, or if a RUN_CONTINUOUS schedule table is interrupted by another RUN_CONTINUOUS schedule table, a schedule switch will take place. When the first schedule table is switched back it will start from the beginning and not resume from the interrupted location. LinIfResumePosition is ignored.

1.4.2 LinIf_CheckWakeup

The implementation of the LinIf_CheckWakeup API is based on the AR4.0.3 specification.

1.4.3 LinIf Master and Slave Configurations

The LinIf configuration works only as a pure master or a pure slave; combinations of both master and slave configurations are not supported. When in a pure slave channel configuration, it is recommended that each hardware channel is config-

uredfor unique LinIf channels.

LinTp should be enabled to make use of LinSlave functionalities.

LinSlave is not yet supported for Lin Drivers.

1.4.4 Node Address Reconfiguration and Auto Addressing

NAD reconfiguration and Auto Addressing cannot be used together on the same LIN Channel/Schedule.

1.5 Exclusions

The following API functions are not currently implemented:

- `LinIf_EnableBusMirroring`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- LinIf/LinIfGeneral/LinIfBusMirroringSupported
- LinIf/LinIfGeneral/LinIfMultipleDriversSupported
- LinIf/LinIfGeneral/LinIfMultipleTrcvDriverSupported
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfCddRef
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfPduDirection/LinIfSlave-ToSlavePdu
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfMaster/LinIfJitter
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfLinProto-colVersion
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfRespon-seErrorSignal
- LinIf/LinIfGlobalConfig/LinIfChannel/LinIfScheduleTable/LinIfEntry/LinIfColli-sionResolvingRef

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.1.1 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfBusIdleTimeoutPeriod

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

1.6.1.2 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfFrameId

[ADDED] Default Value	false

1.6.1.3 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfFrameIndex

[ADDED] Default Value	false

1.6.1.4 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfFrameType

[RTA-BSW] Enumeration Literals	ASSIGN, ASSIGN_FRAME_ID_RANGE, ASSIGN_NAD, READ_BY_IDENTIFIER, AUTO_ADDRESSING, CONDITIONAL, EVENT_TRIGGERED, FREE, MRF, SAVE_CONFIGURATION, SPORADIC, SRF, UNASSIGN, UNCONDITIONAL
[AUTOSAR] Enumeration Literals	ASSIGN, ASSIGN_FRAME_ID_RANGE, ASSIGN_NAD, CONDITIONAL, EVENT_TRIGGERED, FREE, MRF, SAVE_CONFIGURATION, SPORADIC, SRF, UNASSIGN, UNCONDITIONAL

1.6.1.5 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfPduDirection/LinIfTxPdu/LinIfTxPduId

[ADDED] Default Value	false

1.6.1.6 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfFrame/LinIfSubstitutionFrames/LinIfFramePriority

[ADDED] Default Value	false

1.6.1.7 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfMainFunctionPeriod

[ADDED] Default Value	false

1.6.1.8 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfMaxFrameCnt

[ADDED] Default Value	false

1.6.1.9 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfNodeConfigurationIdentification/LinIfConfiguredNAD

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.10 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/Lin-

lfNodeConfigurationIdentification/LinIfFunctionId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.11 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfNodeConfigurationIdentification/LinIfInitialNAD

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.12 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfNodeConfigurationIdentification/LinIfNasTimeout

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.13 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfNodeConfigurationIdentification/LinIfSupplierId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.14 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfNodeType/LinIfSlave/LinIfNodeConfigurationIdentification/LinIfVariantId

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1
[ADDED] Default Value	false

1.6.1.15 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfScheduleTable/LinIfEntry/LinIfDelay

[ADDED] Default Value	false

1.6.1.16 LinIf/LinIfGlobalConfig/LinIfChannel/LinIfScheduleTable/LinIfEntry/LinIfEntryIndex

[ADDED] Default Value	false

1.7 API Definition

Table 1.44.LinIf Supported API functions

`LinIf_Init`	Initializes the LIN Interface.
`LinIf_GetVersionInfo`	Returns the version information of this module.
`LinIf_Transmit`	Indicates a request.
`LinIf_ScheduleRequest`	Requests a schedule table to be executed.
`LinIf_GotoSleep`	Initiates a transition into the Sleep Mode on the selected channel.
`LinIf_Wakeup`	Initiates the wakeup process.
`LinIf_SetTrcvMode`	Sets the given LIN transceiver to the given mode.
`LinIf_GetTrcvMode`	Returns the actual state of a LIN Transceiver Driver.
`LinIf_GetTrcvWakeupReason`	Returns the reason for the wakeup that has been detected by the LIN Transceiver Driver.
`LinIf_SetTrcvWakeupMode`	Enables disables and clears the notification for wakeup events on the addressed network.
`LinIf_CheckWakeup`	Called when the EcuM has been notified about a wakeup on a specific LIN channel.
`LinIf_WakeupConfirmation`	Called by the LIN Driver or LinTrcv to report the wake up source after successful wakeup detection.
`LinIf_HeaderIndication`	Called by the LIN Driver to report a received LIN header.
`LinIf_RxIndication`	Called by LIN Driver to report successful response and provides reception data to LIN Interface.
`LinIf_TxConfirmation`	Called by the LIN Driver to report a successfully transmitted response.
`LinIf_LinErrorIndication`	Called by the LIN Driver to report a detected error event during header or response processing.
`LinIf_GetPIDTable`	Retrieves all assigned PID values. The order is congruent to the LIN frame index.
`LinIf_SetPIDTable`	Sets all assigned PID values. The order is congruent to the LIN frame index
`LinIf_GetConfiguredNAD`	Reports the current configured NAD.
`LinIf_SetConfiguredNAD`	Sets the current configured NAD.

For further information on these functions, please refer to AUTOSAR_SWS_LINInterface.pdf

1.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

1.8.1 Configuring LinIfDriverRef

When configuring LinIf using ISOLAR-AB, the LinIfDriverRef parameter can be generated correctly by ConfGen, but it cannot be configured manually using the BSW Editor. As illustrated in the figure below, no reference suggestion by the tool is available:

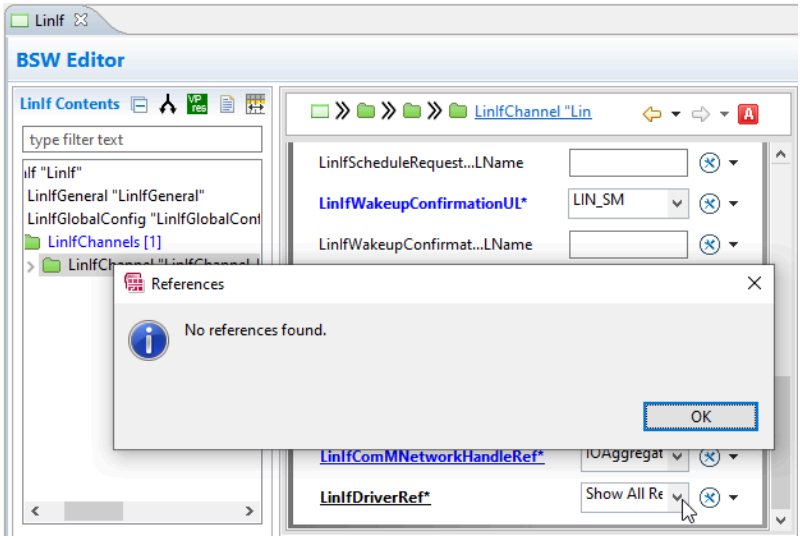

Figure 1.2.**LinifDriverRef** manual configuration limitation

This problem can be circumvented by using a different ISOLAR-AB editor, as described in these steps below:

- In the **Filesystem Navigator**, use the **AUTOSAR Example Editor** to open the file that contains the LinIf EcucValue (in our example, Project_EcucValues.arxml includes the LinIf EcucValue).

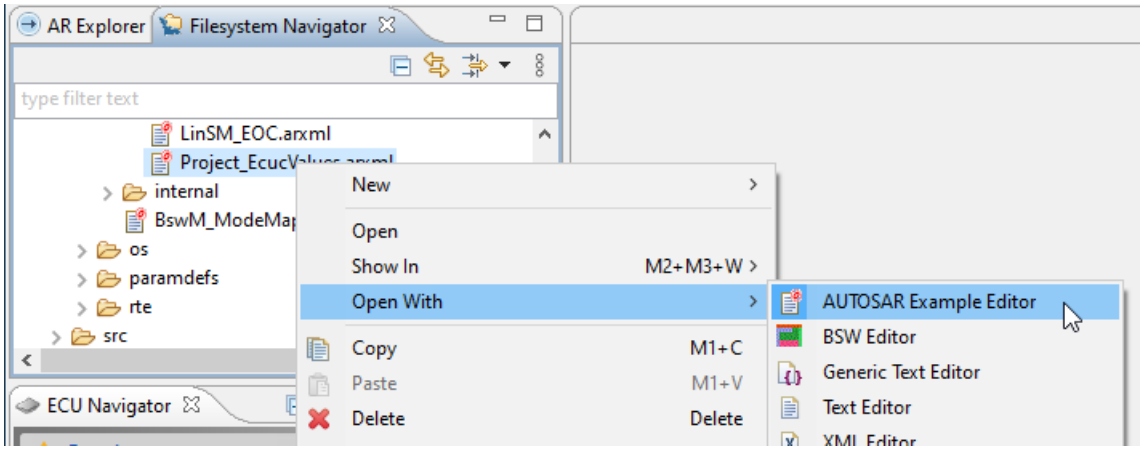

Figure 1.3.Open with AUTOSAR Example Editor.

- In the **AUTOSAR Contents Tree**, expand **LinIf/LinIfGlobalConfig/LinIfChannel** to reveal a list of the Ecuc Reference Values.

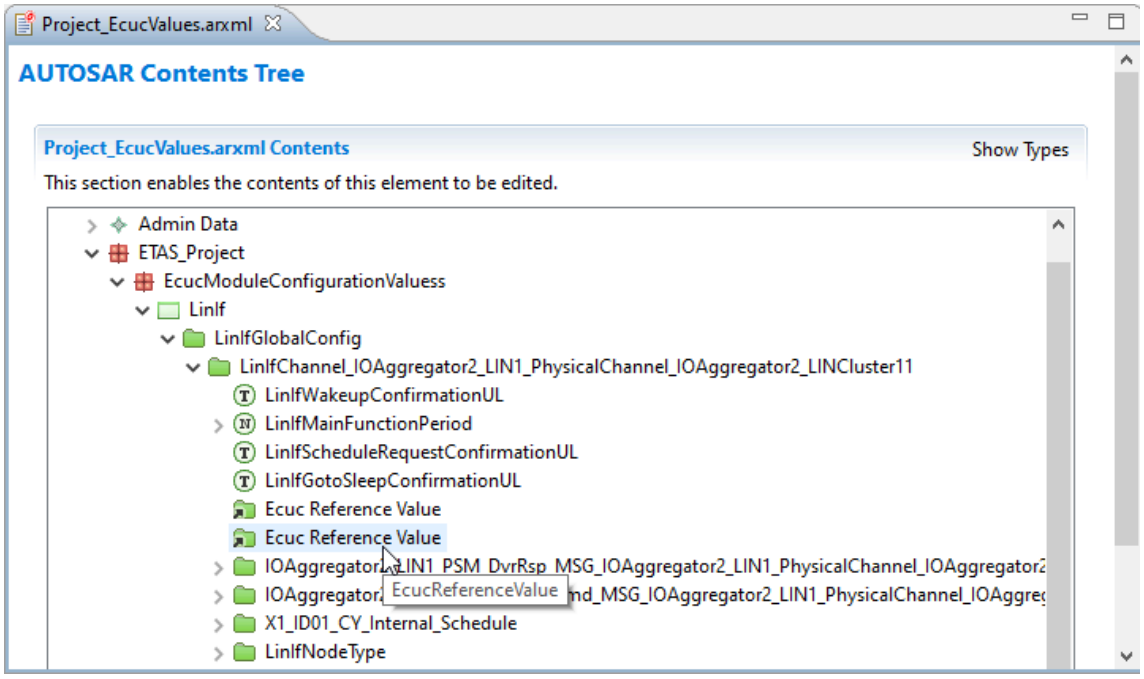

Figure 1.4.The list of the **Ecuc Reference Value**

- Right click **LinIfChannel** and select **New Child/Reference Values/Ecuc Reference Value** to create a new Ecuc Reference Value.

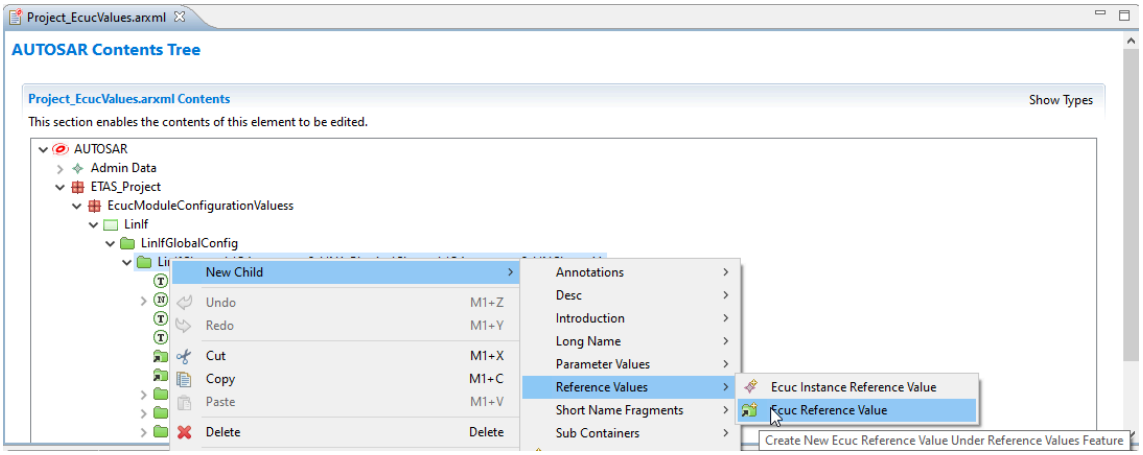

Figure 1.5.Create new **Ecuc Reference Value**

- Once the new Ecuc Reference Value has been created, right click it and select **Show Properties View** to open **Properties Tab**.
- The **Properties Tab** will appear as shown below.

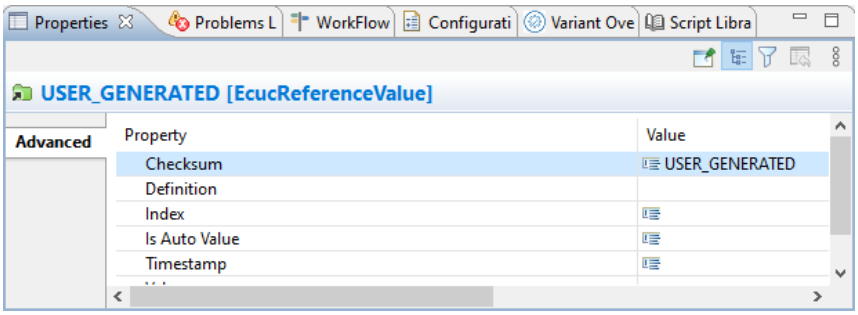

Figure 1.6.**Properties Tab**

- In the **Properties Tab**, click on the Value of the **Definition** property to show the list of the paramdefs, and then choose **LinIfDriverRef**...

Figure 1.7.List of paramdefs

- The Value of **Definition** will be updated.

- When you click on the Value of the **Value Property** it will show the List of Values that can be referenced.

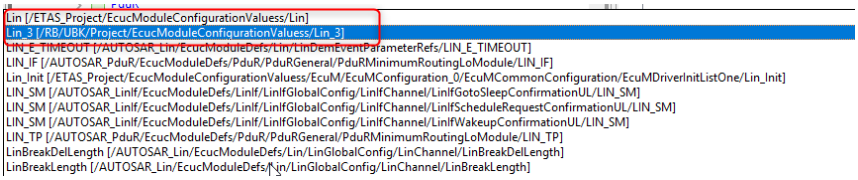

Figure 1.8.List of Values can be referenced

- You can now select which Lin module you want to be referenced for **LinIf-DriverRef** (in the example above there are two available: **Lin** and **Lin_3**).

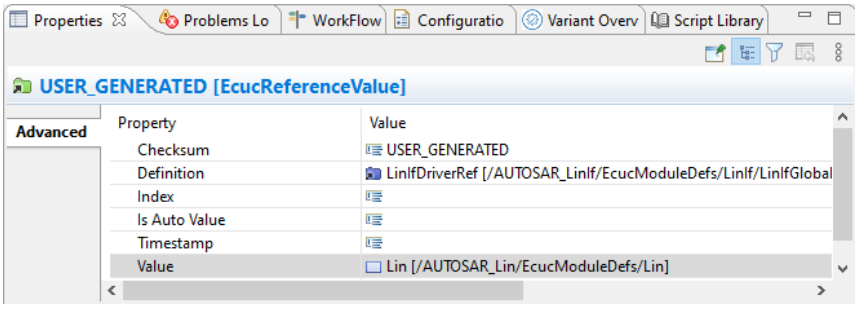

Figure 1.9.**LinIfDriverRef** EcucValue is created

- When you click **Save-All**, the result will be like the example below.

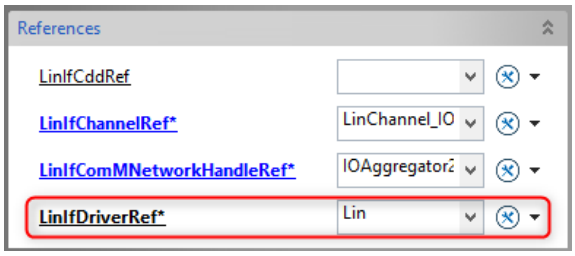

Figure 1.10.**LinIfDriverRef** is configured.

1.8.2 LinIfFixedFrameSdu

For Node Configuration service type frames, LinIfFixedFrameSdu container should be configured.

1.8.3 LinIfPublicCddHeaderFile

LinIfUserTargetTxPduId and LinIfUserTargetRxPduId are not used to generate Pdu Id for Tx and Rx Pdus with CDD as upper layer.

LinIfPublicCddHeaderFile defines header files for callback functions which shall be included in case of CDDs. LinIfPublicCddHeaderFile can be configured with the CddComIfUpperLayer name to get the target Tx id.

1.8.4 LinIfUserTxUL

This parameter should be configured as either PDUR or CDD. If CDD is used, then LinIfTxConfirmationUL and LinIfTxTriggerTransmitUL must be used to set the names of the relevant callbacks in the CDD layer.

1.8.5 Multiple Channels

If multiple channels are used in the LinIf configuration, the handle IDs for the PDUs will be unique across all the channels taken from Com. The maximum number of LinIf channels that can be supported in the overall configuration is 254.

1.8.6 LinSlave

Only Unconditional, MRF, SRF and event-triggered frames are supported as per AUTOSAR specifications.

In configurations where node configuration services are used, the buffer LinIf_SlaveNodeConfig_as (of type LinIfSlave_NodeCfgSrv_tst) holds the updated PID list and NAD values. LinIf_SlaveNodeConfig_as should be mapped to a NVRAM memory section by the user to prevent the loss of the updated values upon HW reset. If the buffer is not stored in NVRAM upon HW reset, the NAD and PID values will have to be updated again via the node configuration services.

In configurations, the callout LinIfResponseErrorSignalChangedCallout is supported to provide callout when LinIf_LinErrorIndication is called with a valid error ID.

The signal LinIfResponseErrorSignal is not supported in LinIf and the user can define the call for Com_SendSignal inside the LinIfResponseErrorSignalChanged-Callout function to indicate the response error.

1.8.6.1 LinIf_LinTp

LinTp specific lock sections will now use LinIf_LinTp as the naming due to the new approach for exclusive area usage.

1.8.6.2 Lin_GeneralTypes Template

Lin_GeneralTypes.h_tpl is AR compliant that can be used by default by RTA-CAR, and Lin_GeneralTypes.h_ETAS_tpl retaining RB type definitions that can be used for ETAS drivers.

1.8.6.3 LinTpScheduleChangeDiag

The configuration parameter “LinTpScheduleChangeDiag” is required to be set to true in order to transition the schedule table from a functional request to an SRF (if the previous schedule table was SRF), and subsequently to the application schedule table.

Therefore the configuration parameter “LinTpScheduleChangeDiag” should be

activated and to ensure BswM_LinTP_RequestMode() support in BswM if there are any functional LinTp request use cases. Otherwise the project has to take care of the physical to application schedule switch.

1.8.7 Configuration of Non-AUTOSAR Features

The following should be noted with regard to the configuration of some of LinIf's non-AUTOSAR features.

1.8.7.1 LinIfidCalibrationSupport

The LinIfidCalibrationSupport extension parameter (in /Lin/LinIfGeneral) enables/disables the calibration of Frame ID (the ID for Lin Frames is provided by the calibration tool).

When calibration is enabled, the initial value of the calibration variable will be the ID of the frame configured.

Calibration is possible only for the Frame ID of Unconditional frames, it is not possible to calibrate the Frame ID of Diagnostic or Sporadic frames.

The name of the Calibration parameter for the corresponding frame is LinIf_id<LinIfFrameName>LinId_C, so the change in the ID value of the corresponding frame results in the change of the Pid of the respective frames.

The name of the Lin frame cannot be changed through calibration.

When LinIfidCalibrationSupport is configured as TRUE, then SchM_LinIf.h should be included in LinIf_Cfg_SchM.h.

1.8.7.2 LinIfRbDiagMonHookSupport

LinIfRbDiagMonHookSupport should be set to TRUE for a LinIf channel for MonHook notification to MRF and SRF.

1.9 Integration Advice

1.9.1 Init Functions

LinIf is initialised by LinIf_Init.

1.9.2 DeInit Functions

LinIf_DeInit deinitializes LinIf.

1.9.3 Scheduled Functions

The AUTOSAR specification defines one instance of LinIf_MainFunction per channel, named LinIf_MainFunction_<LinIfChannel.ShortName>. In RTA-BSW, all of these instances call a single LinIf_MainFunction with the channel number passed as an input parameter.

LinIf_MainFunction is the central processing function of LinIf. Its main tasks include polling the Schedule Table Manager, initiating frame transmission, resolving transmissions and interacting with upper and lower layers.

LinIf_MainFunction should be called periodically, as controlled by the LinIfMain-

FunctionPeriodconfiguration parameter. The tick is used as the smallest time entity in the scheduling of communication, LinIf does not consider the jitter.

LinIf_MainFunction and LinSM_MainFunction should be scheduled together.

Scheduled callback functions are only invoked in the LinIf_MainFunction and not in the LinIf_ScheduleRequest function. This is because LinIf provides the callback only when the schedule switch happens.

1.9.4 Post-Build Support

Post-Build support is provided in the LinIf module.

LinIf channels and Lin channels do not support Post-Build Variant.

The multiplicity for the LinIfScheduleTable container cannot be changed at the container level because of the reference to the LinSM schedule, which does not support Post-Build Variant.

A project with one or more top level Post-Build Variants, in combination with EcuMModuleParameter configured with NULL_PTR, is an invalid configuration.

If the project has no top level Post-Build Variants, and the parameter EcuMModuleParameter is configured with POSTBUILD_PTR, then LinIf_Init is called with a Post-Build pointer. In this case, a statically generated configuration structure for PreCompile is initialized during LinIf_Init.

1.9.5 LinIf_MonHook_E_OK, LinIf_MonHook_E_NOT_OK and LinIf_Diag_MonHook

Empty definitions of the LinIf_MonHook_E_OK and LinIf_MonHook_E_NOT_OK extension APIs are available in LinIf_MonitoringHook.c which should be edited to meet project requirements. The value of the Direction parameter is 0 for TX and 1 for RX.

LinIf_Diag_MonHook is reported with Pid, Channel Id and LinIf frame status during a successful or unsuccessful frame transmission or reception.

1.9.6 Lin_WakeupInternal

This API is used in RTA-BSW when Wakeup Source for Lin controller or Lin Tranceiver is configured.

In order to support Lin drivers made for older AUTOSAR versions, Lin_WakeupIntegnInternal is defined in LinIf_Integn.c. When using a Lin driver without Lin_WakeupInternal, Lin_WakeupIntegnInternal should be edited to return E_OK.

1.9.7 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the imple-

mentationof these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The LinIf module uses the following OS resources:

- LinIf_ScheduleTableProtArea_LockNoNest
- LinIf_ChannelStateProtArea_LockNoNest
- LinIf_CheckWakeup_Lock
- LinIf_LockNoNest
- LinIf_LinTp_LockNoNest

1.10 Hardware Requirements

Table 1.45.LinIf Hardware Requirements

Support	Usage
FLASH	14227 bytes
RAM	102 bytes

Usage calculated based on a sample configuration.

2 Lin State Machine

Module name	LinSM
Package	RTA-LIN
AUTOSAR Module ID	141
AUTOSAR Version	22-11

2.1 Introduction

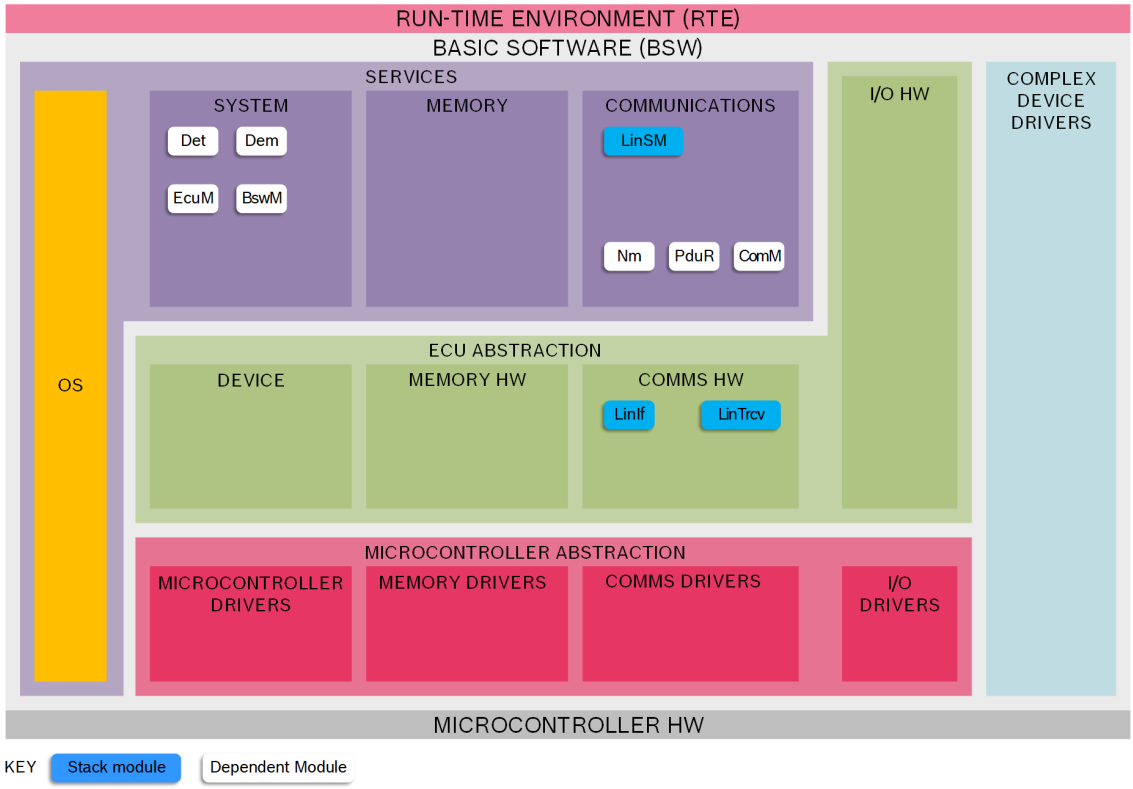

Figure 2.1.RTA-LIN Architecture

The LIN State Manager module (LinSM) is located in the communication services abstraction layer, acting as part of the upper layer for the RTA-LIN stack.

LinSM provides a hardware-independent interface to the low-level communication system, and is primarily used for controlling startup/shutdown and other mode changes for communication channels, as requested by the ComM module.

The diagram above shows the position of LinSM in the AUTOSAR layered architecture.

2.1.1 Interaction and Dependencies

LinIf

LinSM's primary interaction with the lower layer is through the LIN Interface (Lin-

If).LinSM assumes the following primitives are provided by the LinIf module:

- Transmission of the Go-to-Sleep command (LinIf_GotoSleep)
- The wakeup of the Lin bus (LinIf_Wakeup)
- Request to change schedule tables (LinIf_ScheduleRequest)

It is assumed that the LinIf module will call the following callbacks:

- Confirming the Wake Up signal has been transmitted (LinSM_WakeupConfirmation)
- Confirming that the Go-to-Sleep command has been transmitted (LinSM_GotoSleepConfirmation)
- Confirming a schedule change (LinSM_ScheduleRequestConfirmation)

ComM

The upper layer Communication Manager (ComM) requests communication via the LIN stack, and it queries the state of the LinSM module.

BswM

LinSM notifies BswM when a state is changed. BswM interfaces LinSM when requesting a new schedule table.

Module Configuration

This module can be generated by the RTA-BSW Configuration Generator.

For any unsupported automatic configuration, the module can be manually configured by the user.

2.2 Supported Features

The LIN State Manager (LinSM) is responsible for the control flow of the LIN Bus. As such, the main features of LinSM are:

- Controlling the communication status of one or more LIN networks (no communication or full communication).
- Handling communication mode requests.
- Handling schedule changes, switching schedule tables when requested by the upper layer(s).
- Handling go to sleep and wakeup, when requested by the upper layer(s).
- Notification to upper layers of state changes.

2.2.1 The LinSM state machine

LinSM operates in a state-machine. Each connected network operates in an independent sub-state-machine.

2.2.2 Non-AUTOSAR Features

In addition to the AUTOSAR-defined features listed above, LinSM also provides

the following additional functionality.

2.2.3 Schedule Table Queue

The Event Schedule Table can be optionally maintained in a queue. By default, the queue is filled with a NULL schedule table at the zero position (i.e. if no run_continuous schedule table is requested/running, then the default schedule table will be NULL).

When an Event schedule table is requested and there is no Event schedule table already running, an Event schedule table is loaded for execution, otherwise the new Event schedule table is loaded into the queue to be processed later.

Each Event schedule table will carry a priority and will occupy a position inside the queue based on that priority:

- If the newly requested Event schedule table has a priority less than the priority of an already existing schedule table, the newly requested schedule table will take a position after all the higher prioritized schedule tables.
- If the newly requested Event schedule table has a priority higher than the priority of already existing schedule tables, the newly requested schedule table will take a position before all the lower prioritized schedule tables.
- If the newly requested schedule table is already present inside the Event schedule queue, then the newly requested schedule table is once again added with equal priority.

The queue will come into action only when an Event schedule table is running and a new Event schedule table is requested (until then there is no need to queue the schedule tables).

The queue is emptied when all the schedule tables are executed or when either LinSM_Init, LinSM_DeInit or LinSM_RequestComMode (No_Com) is called.

When NULL schedule table is requested, the queue is flushed for that particular LinSM channel.

The Event Schedule Table queue is controlled by the LinSMEnableScheduleTableQueue extension parameter - see the Configuration Advice for more information.

2.2.4 Lin Slave Feature

The LinSM Channel can be configured as either a Master or Slave, note LinSM Slave sleep and wakeup sequences are different from Master.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 LinSM/LinSMConfigSet/LinSMChannel/LinSMChannelIndex

Short Name	LinSMChannelIndex
Description	Index of the controller. To be able to interface to the correct hw.
Multiplicity	1..1
Default Value	-
Type	ECUC-REFERENCE-DEF

2.3.1.2 LinSM/LinSMConfigSet/LinSMChannel/LinSMEnableSchedule-TableQueue

Short Name	LinSMEnableScheduleTableQueue
Description	This parameter enables or disables of queue Handling for Run Once Schedule table per channel.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.3 LinSM/LinSMConfigSet/LinSMChannel/LinSMMaxQUEUESize

Short Name	LinSMMaxQUEUESize
Description	This parameter holds the Maximum queue size
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.4 LinSM/LinSMConfigSet/LinSMChannel/LinSMSchedule/LinSM-ScheduleTablePriority

Short Name	LinSMScheduleTablePriority
Description	This parameter holds the priority of each schedule table
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.5 LinSM/LinSMConfigSet/LinSMChannel/LinSmNetworkIndex

Short Name	LinSmNetworkIndex

Description	This parameter holds the unique channel index value. The value shall be the same as the ComMChannelId of the ComMChannel referenced by LinSMComMNet-workHandleRef. Implementation Type: NetworkHandle-Type
Multiplicity	1..1
Value Range	0..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.2.1 LinSM_DeInit

Syntax	`LinSM_DeInit(void)`
Description	This function Deinitializes the LIN State Machine. It deinitializes all the static variables to their default values, and it sets the LINSM channel state to LINS-M_NO_COM.

2.3.2.2 LinSM_SchedulePositionQueried

Syntax	`LinSM_SchedulePositionQueried(VAR(NetworkHandleType,AUTOMATIC) network,VAR(LinIf_SchHandleType,AUTOMATIC) schedule)`
Parameter In	`network` Network ID of the reference LinSM channel.
Parameter In	`schedule` ID of the schedule table.
Description	Reads the QUEUEd schedule IDs based on the priority of the schedule ID. This function is called only when the queue event is enabled in both LinIf and LinSM. If the schedule id is not in the queue list it returns 0.

2.4 Limitations

The LinSM module does not currently have any limitations.

2.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

2.6 Deviations

2.6.1 AUTOSAR Deviations

The AUTOSAR specification defines one instance of LinSM_MainFunction with no parameters. In RTA-BSW, LinSM_MainFunction accepts a channel number as an input parameter, `void LinSM_MainFunction(uint8 Channel)`.

Additionally, there is one function per channel, named LinSM_MainFunction_<LinSMChannel.ShortName>. These functions have no parameters and call LinSM_MainFunction with the appropriate channel number.

2.6.2 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.2.1 LinSM

[RTA-BSW] Description	Configuration of the Lin State Manager module(with vendor specific enhancements of RB).
[AUTOSAR] Description	Configuration of the Lin State Manager module.

2.6.2.2 LinSM/LinSMConfigSet/LinSMChannel/LinSMSchedule

[RTA-BSW] Introduction	Only applicable to LIN master nodes.As slave node is not yet supported, atleast one schedule should be configured in all nodes.
[AUTOSAR] Introduction	Only applicable to LIN master nodes.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

2.6.2.3 LinSM/LinSMConfigSet/LinSMChannel/LinSMSilenceAfterWakeup-Timeout

[RTA-BSW] Description	Timeout in seconds after a failed wakeup sequence until a new wakeup process is started
[AUTOSAR] Description	Timeout in seconds after a failed wakeup sequence until a new wakeup process is started.

2.6.2.4 LinSM/LinSMConfigSet/LinSMModeRequestRepetitionMax

[RTA-BSW] Description	Specifies the maximal amount of mode request repetitions without a respective mode indication from the LinIf module until the LinSM module reports a Runtime Error to the Det and tries to go back to no communication.

[AUTOSAR] Description	Specifies the maximal amount of mode request repetitions without a respective mode indication from the LinIf module until the LinSM module reports a Development Error to the Det and tries to go back to no communication.

2.7 API Definition

Table 2.13.LinSM Supported API functions

`LinSM_Init`	Initializes the LIN State Manager.
`LinSM_ScheduleRequest`	The upper layer requests a schedule table to be changed on one LIN network.
`LinSM_GetVersionInfo`	Returns the version information of this module.
`LinSM_GetCurrentComMode`	Queries the current communication mode.
`LinSM_RequestComMode`	Requests a communication mode.
`LinSM_MainFunction`	Periodic function that runs the timers of different request timeouts.
`LinSM_ScheduleRequestConfirmation`	LinIf calls this callback when the new requested schedule table is active.
`LinSM_GotoSleepConfirmation`	LinIf calls this callback when the go to sleep command is sent (successfully or unsuccessfully) on the network.
`LinSM_WakeupConfirmation`	LinIf calls this callback when the wakeup signal command is sent (successfully or unsuccessfully) on the network.
`LinSM_GotoSleepIndication`	LinIf calls this callback when the go to sleep command is received on the network or a bus idle timeout occurs.

For further information on these functions, please refer to AUTOSAR_SWS_LIN-StateManager.pdf

2.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

2.8.1 Event Schedule Table Queue

As explained in the Supported Features section, if the LinSMEnableScheduleTableQueue extension parameter is configured as TRUE for any channel, the

Event schedule table will be maintained in a queue.

- At least one RUN_ONCE schedule table should present inside a channel, and the LinSMScheduleTablePriority parameter should be configured for that schedule table.
- If more than one RUN_ONCE schedule table is present, make sure that the LinSMScheduleTablePriority parameter is configured for each of the RUN_ONCE schedule tables, and that the LinSMScheduleTablePriority value is unique across those tables.
- The LinSMMaxQUEUESize parameter should be configured to a value of less than 255.
- The value of the LinSMScheduleTablePriority parameter for any schedule table should be less than the value of the LinSMMaxQUEUESize parameter (and less than 255).

2.8.2 LinSmNetworkIndex

The LinSmNetworkIndex extension parameter (in LinSMConfigSet/LinSMChannel/) holds the unique channel index value. This value is not the same as the ComMChannelId of the ComMChannel, which is referenced by LinSMComMNetworkHandleRef.

2.8.3 LinSMChannelIndex

To be able to interface to the correct hardware, the Index of the controller should be set in the LinSMChannelIndex extension parameter (in LinSMConfigSet/LinSMChannel/).

2.9 Integration Advice

2.9.1 Init Functions

LinSM is initialised by LinSM_Init.

2.9.1.1 LinSM_Init Input Parameter

The AUTOSAR specification for LinSM_Init has an input parameter `void LinSM_Init(const void* configPtr)`. The configPtr parameter is only relevant for the VARIANT-POST-BUILD, and it is ignored for VARIANT-PRE-COMPILE. Only VARIANT-PRE-COMPILE is supported in this implementation of LinSM, so ConfigPtr should always be NULL.

2.9.1.2 LinSM Timers

The AUTOSAR specification states that the LinSM Timers for the timeout of requests should start before calling the LinIf function, but in the RTA-BSW implementation of LinSM/LinIf the timers should be started after the LinIf function is called.

2.9.2 DeInit Functions

LinSM_DeInit deinitializes the LinSM module. It deinitializes all the static variables to their default values and it sets the LinSM channel state to LINSM_NO_COM.

2.9.3 Scheduled Functions

The AUTOSAR specification defines one instance of LinSM_MainFunction with no parameters. In RTA-BSW, the interface follows LinIf instead, with one function per channel named LinSM_MainFunction_<LinSMChannel.ShortName>. All of these instances call a single LinSM_MainFunction with the channel number passed as an input parameter: `void LinSM_MainFunction(uint8 Channel)`.

LinSM_MainFunction and LinIf_MainFunction should be scheduled together

2.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The LinSM module uses the following OS resources:

- LinSM_LockNoNest

2.10 Hardware Requirements

Table 2.14.LinSM Hardware Requirements

Support	Usage
FLASH	2152 bytes
RAM	33 bytes

Usage calculated based on a sample configuration.

3 Lin Transport Protocol

Module name	LinTp
Package	RTA-LIN
AUTOSAR Module ID	62 (Part of LinIf)
AUTOSAR Version	22-11

3.1 Introduction

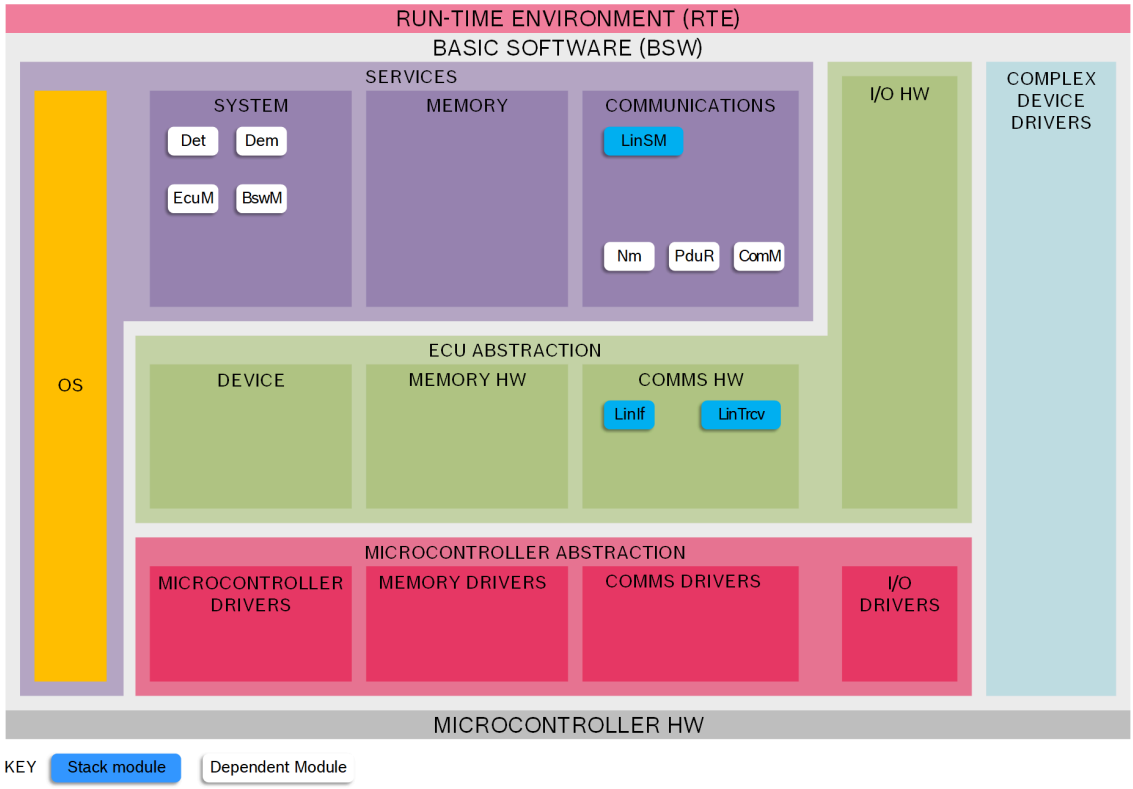

Figure 3.1.RTA-LIN Architecture

LinTp is part of the LinIf module (when enabled via the LinIf configuration parameter LinIfTpSupported) and it is used to handle the segmented transmission and reception of LIN frames.

The diagram above shows the position of LinIf (including LinTp) in the AUTOSAR layered architecture.

3.1.1 Interaction and Dependencies

PduR

LinTp has interaction with the PduR, which handles the TP message buffers either as complete or fragmented messages.

BswM

LinTp connects to the BswM for requesting Schedule Table changes.

Module Configuration

This module's configuration parameters are included in the configuration of another module. See the Configuration Advice section for more details.

3.2 Supported Features

3.2.1 General Summary

LinTp provides Diagnostic Transport layer services to LinIf. See the LinIf section for full details of all the supported features of LinIf. The information below is specific to LinTp.

3.2.2 Response pending frame format

The format of the LinTp Response Pending Frame is: NAD, PCI, 0x7F, SID, 0x78, D1, D2, D3.

3.2.3 Non-AUTOSAR Features in LinTp

In addition to the AUTOSAR-defined features, LinTp also provides the following additional functionality.

3.2.3.1 LinTpInterleavedClientTimer

The LinTpInterleavedClientTimer extension parameter is a monitoring timer for the normal interleaved communication on the bus within diagnostics mode. LinTp starts this timer when the Last MRF frame is successfully placed on the bus.

3.2.3.2 LinTpTimeoutCallbackName

The LinTpTimeoutCallbackName extension parameter provides the name that will be used when generating the external prototype for the function defined by the application layer and used by LinTp for timeout notification.

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 LinTp/LinTpGlobalConfig/LinTpChannelConfig/LinTpInterleaved-ClientTimer

Short Name	LinTpInterleavedClientTimer
Description	Monitoring timer for the normal interleaved communication on bus with-in diagnostics mode and LinTp start this timer when Last MRF frame is placed successfully on bus.
Multiplicity	0..1

Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

3.3.1.2 LinTp/LinTpGlobalConfig/LinTpChannelConfig/LinTpTimeoutCallbackName

Short Name	LinTpTimeoutCallbackName
Description	Function defined by application layer and used by LinTp in order to notify LinTp timeout notification.This parameter name will be used in generating function external prototype.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

`LinTp_CancelReceive`

LinTp does not have any API extensions.

3.4 Limitations

The following Autosar-defined features for LinTp are not supported.

- LinTp timeout callback functions

3.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- LinTp/LinTpGlobalConfig/LinTpChannelConfig/LinTpDropNotRequestedNad
- LinTp/LinTpGlobalConfig/LinTpMaxRxNSduCnt
- LinTp/LinTpGlobalConfig/LinTpMaxTxNSduCnt

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 LinTp/LinTpGlobalConfig/LinTpRxNSdu/LinTpRxNSduId

[RTA-BSW] Multiplicity	0..0
[AUTOSAR] Multiplicity	1..1

3.7 API Definition

Table 3.5.LinTp Supported API functions

`LinTp_Init`	Initializes the LIN Transport Layer.
`LinTp_Transmit`	Requests the transfer of segmented data.
`LinTp_GetVersionInfo`	Returns the version information of this module.
`LinTp_Shutdown`	Shuts down the LIN Transport Layer.
`LinTp_CancelTransmit`	Dummy method introduced for interface compatibility.
`LinTp_ChangeParameter`	Dummy method introduced for interface compatibility.

For further information on these functions, please refer to AUTOSAR_SWS_LINInterface.pdf

3.8 Configuration Advice

There is no specific advice for the configuration of LinTp, but see the LinIf section for configuration advice for LinIf, which includes the LinTp module.

3.9 Integration Advice

3.9.1 Init Functions

LinTp is initialised by LinTp_Init.

LinIf should be initialized before the initialization of LinTp.

3.9.2 DeInit Functions

LinTp_ShutDown is used to shutdown Lin_Tp.

3.9.3 Scheduled Functions

LinIf_MainFunction is the central processing function of LinIf, but there are no scheduled functions for LinTp.

3.9.4 Post-Build Support

Although Post-Build support is provided in the LinIf module, Post-Build Selectable is not specifically supported in LinTp. To use the Post-Build Selectable feature in LinTp, variation points must be set in LinIf.

The Post-Build Variant value for the LinTpChannelRef parameter is FALSE, and the Post-Build Variant multiplicity for the LinTpChannelConfig container is FALSE.

3.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in

the OS with the correct task/ISR allocation.

The LinTp module does not use any exclusive OS resources.

3.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

4 Glossary

CDD
Complex Device Driver

Com
AUTOSAR Communication module

ComM
AUTOSAR Communication Manager

DSM
RTA-BSW internal module (Diagnostic State Manager) fetches the latest DEM error status

LIN Local Internet Network

Lin The LIN Driver module

LinIf
AUTOSAR LIN Interface module

LinSM
AUTOSAR LIN State Manager module

LinTp
AUTOSAR LIN Transport Protocol (part of LinIf)

LinTrcv
AUTOSAR LIN Transceiver Driver module

MRF
Master Request Frame

NAD
Node Address

PCI Protocol Control Information

PduR
PDU Router

SID Service Identifier

SRCF
Slave Response Control Frame

RX Reception

SRF
Slave Response Frame

TP Transport Protocol

TX Transmission